



ИСКУССТВЕННЫЙ ИНТЕЛЛЕКТ

Новый опыт программирования с использованием искусственного интеллекта

Соблазнение ИИ-помощников в написании кода

Стефани Кирмер

18 марта 2026 года 12 минут чтения

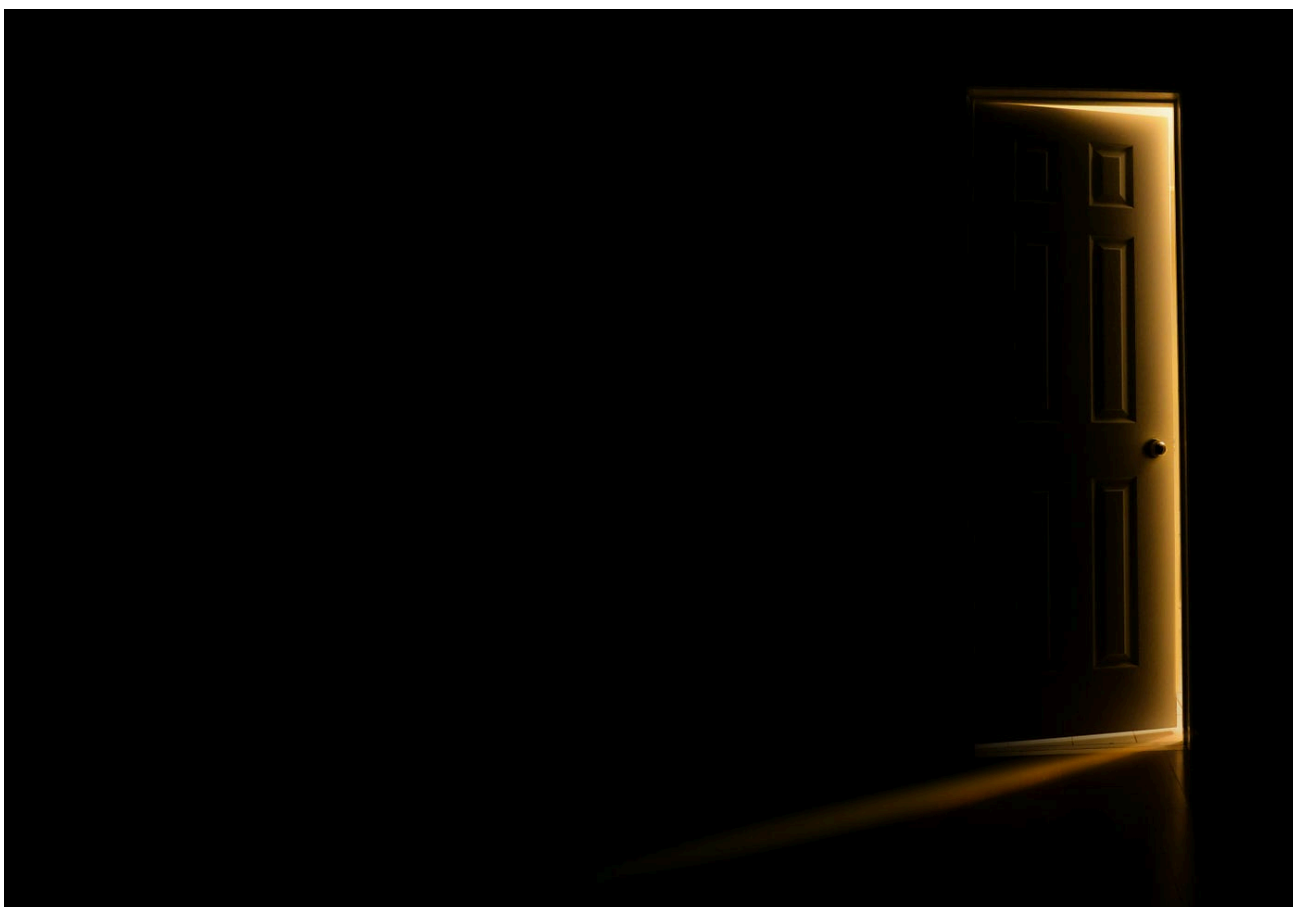


Фото Лань Гао, Unsplash

В июле прошлого года я написал статью о том, как работает поле
разработку программного обеспечения может повлиять

растущая интеграция инструментов для помощи в написании кода на основе больших языковых моделей. К сожалению, я писал эту статью сразу после выхода первой крупной функциональной версии Claude Code. Хотя Claude Code технически существовал уже в феврале 2024 года, только в мае 2025 года он был расширен и стал предлагать более сложные функции помощи в написании кода, которыми он и некоторые другие инструменты обладают. Из-за этого в моих рассуждениях в той статье не были учтены некоторые изменения, произошедшие с тех пор.

Теперь я хочу по-новому взглянуть на ситуацию с использованием инструментов для написания кода на основе больших языковых моделей и понять, на каком этапе мы находимся. В частности, я хочу подумать о том, как эта технология повлияет на нашу работу сейчас и в будущем.

1. Функциональность

О какой сложности я говорю? Что ж, я использовал несколько разных решений для помощи в написании кода (Github Copilot, Claude Code) в своей работе, а также консультировался с разработчиками, которые пробовали другие решения (Cursor, Replit и т. д.). У них разный уровень возможностей, но среди ключевых элементов можно выделить следующие:

- возможность получить доступ ко всем файлам в вашем проекте, искать в них информацию и анализировать их содержимое
- возможность добавлять в проект значительные фрагменты кода или целые файлы
- использование «рассуждающих» больших языковых моделей, которые разбивают задачи на части и обрабатывают их по

отдельности, попутно объясняя пользователю, как происходит обработка

- инструменты-агенты, с помощью которых модели могут самостоятельно обращаться к различным программам для выполнения задач, с которыми большие языковые модели не справляются (включая поиск в интернете)

Ничто из этого не требует изменения нашего понимания больших языковых моделей как сущности и их структуры, но мы дополняем базовую языковую модель функциями, расширяющими некоторые из ее возможностей. «Рассуждающие» языковые модели на самом деле просто используют другие стратегии подсказок и позволяют выполнять несколько параллельных процессов с помощью языковой модели и объединять их. Хотя языковая модель остается тем же строительным блоком, мы комбинируем их по-разному и находим им разное практическое применение, поэтому теперь они более полезны и эффективны при решении конкретной задачи — написании кода.

This isn't meant to diminish the downsides to these tools, or to LLMs in general. I've talked about numerous ways that LLM technology has serious negative externalities. But I don't think we can say, in the narrow space of software engineering, that this technology doesn't work. It's not perfect, clearly — I still get very frustrated when I'm writing code and I ask a code assistant a question and it bungles the whole thing — but the technology we have today is able to serve a useful function.

2. How People Respond

As I talk to friends in the machine learning and software engineering space about this state of affairs, I hear a few different perspectives. Some people are enthusiastically adopting AI code assistants in every way they can. They'll give the tool a prompt and let it write the code, and come back later to review, or have the tool do the review itself. They'll spin up multiple LLMs to collaborate on issues, reviewing each other's work and producing voluminous amounts of code while humans sleep. This is a form of what readers may be familiar with as "vibe coding". For these people, being freed from writing code themselves is an unalloyed good, and they're thrilled by the productivity increases they can achieve. Writing code, for them, was always mainly a means to an end, and they don't mind dispensing with that labor. They are producing new software at speeds never before anticipated, and by and large, it's meeting their needs.

On the other hand, there are those who I think of as "craftspeople". These are developers and engineers who have a love for the work of thinking about code and writing code, and enjoy the journey as much as the destination, if not more. For these people, the advent of AI code assistants is deeply troubling. When you enjoy your work because it requires thoughtfulness, creativity, and resilience, and you take pleasure in the hard work, it's alarming to be faced with a new paradigm suggesting that none of these skills on your part are necessary or desirable. Some of the most talented and skilled software engineers I know have talked about wanting to quit the whole profession rather than be pushed into a vibe-coding paradigm in their day to day work, where prompting and reading code reviews constitute the bulk of their responsibilities.

[Nicki Boykis's latest piece addresses this thoughtfully](#)– her advice for those of us feeling depressed about the direction of our field

is to redouble our efforts to find ways to scratch the itch of wanting to be creative and make meaning in our work. I appreciate the value she places on these skills and feelings, but it does suggest that even she doesn't see the actual job keeping the core character we have become accustomed to.

This theory is of course a spectrum, populated with people who may enjoy coding a bit, but are all right with handing off most of that work, or people who really like to code, but recognize that business pressures require they adapt their processes to include more AI. Wherever you land, many if not most of us are concerned about how this shift is going to affect our careers and job prospects, as well as the state of the software engineering field as a whole.

The Seduction

But what is it we are really experiencing? What is it like sitting down in front of your keyboard and spinning up your IDE in this new era? There's something strangely seductive about having a little tool on the side of your screen that can just handle a task for you.

You know that the assistant can probably write the next function you need to add to your code. Even if you haven't used it yourself, you've heard your peers rave about its abilities. And, what's the downside, anyway? Why not just go for the code assistant and have it do a little task?

You might have concerns about job security — are you going to become obsolete as tools like this increase their capability or we find more effective ways to use them? Will you lose the skills at you've earned over the course of your career, as you stop

using them on a daily basis in favor of letting the AI do tasks? Nobody can tell you if these are real concerns, because we just don't know for sure yet how the workplace for software engineers is going to evolve over the longer term.

You may also be aware of broader implications of generative AI. You're implicitly saying, "this work that I need done is worth the negative costs of this technology." By choosing to click that code assistant chat button, you are deciding that your use case is worth the electricity. This is worth the water usage. This is worth supporting and boosting an industry and the technology that is, in other areas, responsible for significant social, political, and cultural negative impacts. You're saying, "I think that's all worth it for me to get a tool to write the code I need to complete this project."

But even when you do have these tradeoffs brought to your attention, it's still hard. You're sitting there looking at your code, and part of you says, "I could just do this. I could write this component of this code. I know how to write this function." But you've got this little bug, this little itch in the form of a chat window on the side of the screen or a terminal command just waiting. "It'll take me 3 hours to write this class and get it working and write the tests. But man, I could just push that button. That button's just right there. Push that button, and this will be done in a few minutes, and then I can move on to the next thing. It might even work better than what I'd write. My boss will be happy. I could be making progress and moving forward, so why not just make the AI tool do the work?"

There are many reasons why bouncing around in your head, because you know about the costs of using this technology, but at seductiveness is still there. Rationalizing starts in — you may

ask yourself, “well, does my single usage of this really make any difference? I’m just one user, after all.” This is a reasonable question to ask, of course. How much difference can one prompt make? Your one prompt really isn’t that resource intensive, and others around the world are using this technology much more for much less worthy endeavors.

On the other hand, one prompt is probably never just one — what if you’re heading down a slippery slope where this becomes a routine part of your work? If your skills atrophy, will that make you more dependent on the tool?

Is this even really up to you any more? Does it feel like you can continue working in software engineering and not pick up these tools? It’s very plausible that maintaining productivity and relevance at work requires you to keep using the code assistant tools. Is it your personal responsibility to hold back the tide of AI code tools, in the face of crowds who eagerly adopt this technology for every possible use case? In a trade off between principled avoidance of technology that has negative social effects, and continuing to be able to feed your family, what is an individual supposed to do? For most of us, material survival has to win out.

3. What Now?

This mental space is a hard place to operate from. We’re witnessing a significant change in how our work is done, and each of us is deciding how we adapt to it. For many, it is emotionally taxing to see the field changing so dramatically, facing the uncertainty about what this means for us and the world around us.

What did our forebears in the earliest days of computer programming think this field was going to look like in the future? In, say, the 1960s, when people were operating mainframes as big as a room and writing code with punch cards, could they have envisioned the Python open source ecosystem? This is kind of how I think about the scale of change that is potentially possible for us now, and it may happen at a rapid pace.

The AI code assistants seem to be here to stay, in some form or another. The larger economic future of the big players in LLMs may be precarious, for reasons I have written about before, but that doesn't necessarily prevent us from having access to some kinds of code assistant tooling, through open source LLMs and tools like <https://amrcode.com/>, <https://opencode.ai/>, or <https://www.tabbyml.com/>. If the models never get any better than they are today, then they're still going to be functionally useful.

Our jobs are going to change, because these new tools are available, and we have to find out how we will evolve. I don't believe our jobs are going to disappear, they are just going to change. We are going to become accustomed to using AI assistants in our coding, and it remains to be seen what the day to day works looks like as a result. Will institutional inertia limit the amount of change we see in our workplaces? Will there still be any place for creativity and craftsmanship in software development and coding? In workplaces, people are already being given performance reviews based on whether they use AI enough to please management, so we don't have much time to think about it.

On a personal level, how are we going to come to grips with the ethical implications of our participation in this industry, and the

ways they are changing? Nobody can answer that for you, of course. Some people may very well quit and change careers, while others will find a way to live with the new paradigm.

We are in a specific bind between what the economy and material conditions expect or demand from us, and the ethical implications of those demands. The vast majority of us need to support our families and aren't in a position to refuse to comply. I think a lot of us are going to have to cope with a cognitive dissonance about these two sides.

Awareness and consciousness of the costs of our system are important, even if they cause us discomfort. Pretending the problems with generative AI don't exist isn't a solution. As social scientists know, honestly interrogating the dynamics, flaws, and power structures of the system we find ourselves in is a prerequisite for improving that system, however incrementally. We can't put the generative AI genie back in the bottle, but we also don't necessarily have to accept the worst case scenario in social, cultural, environmental, and political effects either. Structural change, not individual choice, is the only way to meaningfully improve systems, and if we're informed about the ethical problems we can participate in systemic pushes toward improvement.

• • •

Read more of my work at www.stephaniekirmer.com. I am also speaking at ODSC East at the end of April 2026, on the topic of evaluation strategies for LLM development.

• • •

WRITTEN BY

Stephanie Kirmer

See all from Stephanie Kirmer

Agentic Ai

AI coding assistants

Claude Code

Editor's Picks

Programming

Share This Article



Towards Data Science is a community publication. Submit your insights to reach our global audience and earn through the TDS Author Payment Program.

Write for TDS

Related Articles

ARTIFICIAL INTELLIGENCE

What Do Large Language Models “Understand”?

A deep dive on the meaning of understanding and how it applies to LLMs

Tarik Dzekman

August 21, 2024 31 min read

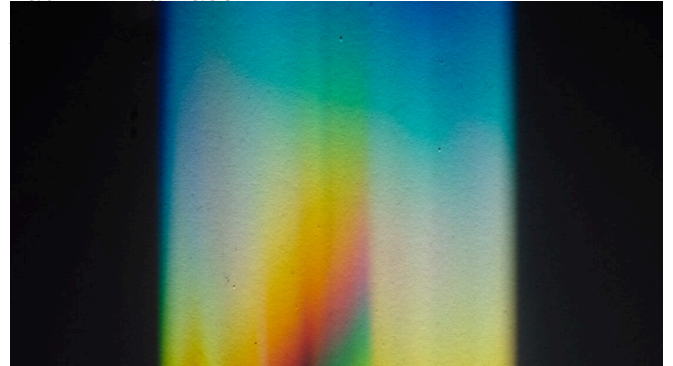


DATA SCIENCE

Optimizing Marketing Campaigns with Budgeted Multi-Armed Bandits

With demos, our new solution, and a video

Vadim Arzamasov



DEEP LEARNING

Deep Dive into LSTMs & xLSTMs by Hand 🖋️

Explore the wisdom of LSTM leading into xLSTMs - a probable competition to the present-day LLMs

Srijanie Dey, PhD

July 9, 2024 13 min read

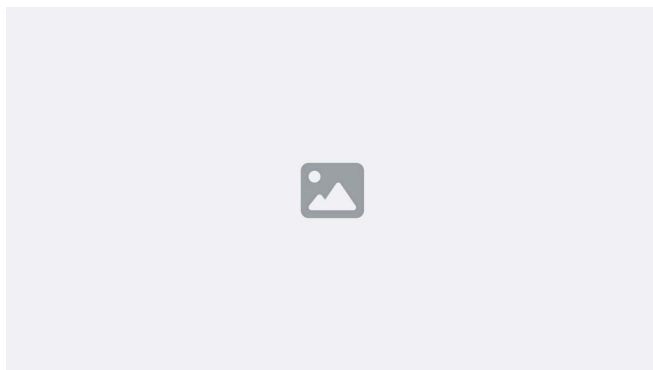
MACHINE LEARNING

Understanding Predictive Maintenance - Wave Data: Feature Engineering (Part 2 Spectral)

Feature Engineering of spectral data

Marcin Stasko

December 1, 2023 13 min read



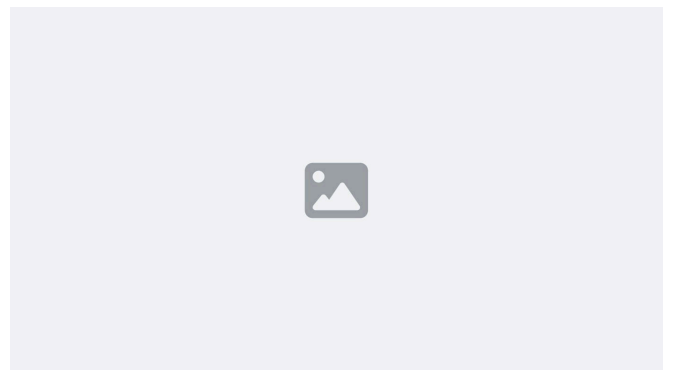
DATA SCIENCE

Latest picks: Data Quality for Everyday Analysis

Your daily dose of data science

TDS Editors

November 17, 2020 1 min read



DATA SCIENCE

Latest picks: No Free Lunch with Feature Bias

Your daily dose of data science

TDS Editors

January 25, 2021 1 min read



DATA SCIENCE

Latest picks: Why & How to use the Naive Bayes algorithms in a regulated industry with sklearn

Your daily dose of data science

TDS Editors

December 2, 2020 1 min read



Your home for data science and AI. The world's leading publication for data science, data analytics, data engineering, machine learning, and artificial intelligence professionals.

© Insight Media Group, LLC 2026

[Subscribe to Our Newsletter](#)

[WRITE FOR TDS](#) · [ABOUT](#) · [ADVERTISE](#) · [PRIVACY POLICY](#) · [TERMS OF USE](#)